Easy Zig (한국어판)

JW Lee

Easy Zig

Zig 프로그래밍 언어 초보자를 위한 종합 가이드

저자*: JW Lee*

서문

Easy Zig 에 오신 것을 환영합니다! 이 책은 Zig 프로그래밍 언어를 처음부터 배우고자 하는 초보자를 위해 설계되었습니다. Python, JavaScript, C 에서 왔든, 이것이 첫 번째 프로그래밍 언어이든, 이 책은 단계별로 안내해 드립니다.

Zig 란 무엇인가?

Zig 는 Andrew Kelley 가 2016 년에 만든 현대적인 시스템 프로그래밍 언어입니다. “더 나은 C”가 되도록 설계되었으며, 메모리와 하드웨어에 대한 저수준 제어를 제공하면서 C 와 C++ 를 어렵고 오류가 발생하기 쉽게 만드는 많은 함정을 피합니다.

```
┌──────────────────────────────────────────────────────────┐
│                 프로그래밍 언어 스펙트럼                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  고수준                                     저수준       │
│  (쉬움)                                    (빠름)        │
│                                                          │
│  Python ─── JavaScript ─── Go ─── Zig ─── C ─── ASM      │
│     │           │           │       │      │      │      │
│  스크립팅      웹 앱      서비스   시스템   OS   하드웨어  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

왜 Zig 를 선택해야 할까?

1. 단순성

C++ 와 달리 Zig 는 숨겨진 제어 흐름, 숨겨진 메모리 할당, 연산자 오버로딩이 없습니다. 보이는 것이 전부입니다.

2. 가비지 컬렉션 없는 안전성

Zig 는 컴파일 시간에 많은 버그를 잡습니다. 기본적으로 null 포인터가 없고, 배열 경계 검사가 있으며, 명시적 오류 처리가 있습니다.

3. 성능

Zig 는 C 및 C++ 와 비슷한 고도로 최적화된 기계어 코드로 컴파일됩니다. Uber 와 같은 회사에서 프로덕션에 사용됩니다.

4. C 상호운용성

Zig 는 C 헤더 파일을 직접 가져와서 사용할 수 있습니다. 바인딩이 필요 없습니다. 이는 수십 년간의 C 라이브러리에 액세스할 수 있음을 의미합니다.

5. Comptime (컴파일 시간 실행**)**

Zig 는 컴파일 시간에 코드를 실행할 수 있어 매크로나 템플릿 없이 강력한 메타프로그래밍이 가능합니다.

```
┌──────────────────────────────────────────────────────────┐
│                      Zig의 주요 기능                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────┐    ┌──────────┐    ┌──────────┐          │
│   │ GC 없음  │    │ C 호환성 │    │ Comptime │          │
│   │──────────│    │──────────│    │──────────│          │
│   │ 수동     │    │ .h 파일  │    │ 컴파일   │          │
│   │ 메모리   │    │ 직접     │    │ 시간에   │          │
│   │ 제어     │    │ 임포트   │    │ 코드 실행│          │
│   └──────────┘    └──────────┘    └──────────┘          │
│                                                          │
│   ┌──────────┐    ┌──────────┐    ┌──────────┐          │
│   │ 안전성   │    │  빠름    │    │ 크로스플랫폼│        │
│   │──────────│    │──────────│    │──────────│          │
│   │ 컴파일   │    │ LLVM     │    │ 모든     │          │
│   │ 시간     │    │ 최적화   │    │ 타겟으로 │          │
│   │ 검사     │    │ 코드     │    │ 컴파일   │          │
│   └──────────┘    └──────────┘    └──────────┘          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Zig 설치하기

설치

macOS (Homebrew):

```
brew install zig
```

Linux:

```
# ziglang.org 에서 다운로드하고 압축 해제
wget https://ziglang.org/download/zig-linux-x86_64.tar.xz
```

```
tar -xf zig-linux-x86_64.tar.xz
export PATH=$PATH:./zig-linux-x86_64
```

Windows: https://ziglang.org/download/ 에서 설치 프로그램 다운로드

설치 확인

```
zig version
```

첫 번째 명령

```
# hello.zig 파일을 만들고 실행:
zig run hello.zig
```

이 책 사용 방법

각 장은 하나의 개념을 다음과 함께 소개합니다: 1. 설명 - 개념이 무엇이며 왜 중요한지 2. 다이어그램 - 작동 방식의 시각적 표현 3. 코드 예제 - 직접 실행할 수 있는 작동하는 코드 4. 출력 - 코드 실행 시 보게 될 내용 5. 핵심 포인트 - 중요한 내용 요약

Zig 여정을 시작해 봅시다!

---

1 장: Hello World

모든 프로그래밍 여정은 Hello World 로 시작합니다. 이 간단한 프로그램은 Zig 프로그램의 기본 구조를 가르쳐 줍니다.

프로그램 구조 이해하기

```
┌──────────────────────────────────────────────────────────┐
│                    Zig 프로그램 구조                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│    ┌──────────────────────────────┐                      │
│    │  1. 임포트                    │                      │
│    │     const std = @import("std");│                    │
│    └──────────────────────────────┘                      │
│                      │                                   │
│                      ▼                                   │
│    ┌──────────────────────────────┐                      │
│    │  2. 상수 & 전역 변수          │                      │
│    │     const MAX = 100;          │                      │
│    └──────────────────────────────┘                      │
│                      │                                   │
│                      ▼                                   │
│    ┌──────────────────────────────┐                      │
│    │  3. 함수들                    │                      │
│    │     fn helper() { ... }       │                      │
```

```
│    └──────────────────────────────────┘                    │
│                        │                                    │
│                        ▼                                    │
│    ┌──────────────────────────────────┐                    │
│    │  4. MAIN 함수 (진입점)            │                    │
│    │     pub fn main() void { ... }   │                    │
│    └──────────────────────────────────┘                    │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

첫 번째 프로그램

```zig
// ============================================================
// 1 장: Hello World - 첫 번째 Zig 프로그램
// ============================================================

// 1 단계: 표준 라이브러리 임포트
// @import 함수는 외부 코드를 가져옵니다
// "std" 는 유용한 함수들이 있는 Zig 의 표준 라이브러리입니다
const std = @import("std");

// 2 단계: main 함수 정의
// - "pub" 은 public (외부에서 호출 가능)
// - "fn" 은 함수를 선언합니다
// - "main" 은 프로그램이 시작되는 진입점입니다
// - "void" 는 이 함수가 아무것도 반환하지 않음을 의미합니다
pub fn main() void {
    // 3 단계: 콘솔에 출력
    // std.debug.print() 는 터미널에 텍스트를 출력합니다
    // \n 은 새 줄을 만듭니다
    // .{} 는 형식 인자를 위한 빈 튜플입니다
    std.debug.print("Hello, World!\n", .{});
}
```

출력:

```
Hello, World!
```

각 부분 분석

Import 문

```zig
const std = @import("std");
```

이 줄은 세 가지를 수행합니다: 1. `@import("std")` - Zig 의 표준 라이브러리를 로드합니다 2. `const std` - `std` 라는 이름의 상수를 생성합니다 3. `=` - 라이브러리를 상수에 할당합니다

다음과 같이 생각하세요:

```
┌─────────────────────────────────────────────────────────────┐
│                        @import("std")                       │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│    표준 라이브러리 내용:                                      │
```

```
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│   │   debug      │  │    mem       │  │    fmt       │          │
│   │  .print()    │  │  .copy()     │  │  .format()   │          │
│   └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                  │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│   │    heap      │  │    fs        │  │    os        │          │
│   │ .allocator()│  │ .openFile()│  │ .system()   │          │
│   └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Main 함수

```zig
pub fn main() void {
    // 여기에 코드
}
```

- `pub` - 함수를 런타임에 보이게 합니다
- `fn` - 함수를 선언하는 키워드
- `main` - 프로그램의 진입점으로 인식되는 특별한 이름
- `()` - 빈 괄호는 매개변수가 없음을 의미
- `void` - 반환 타입 (아무것도 반환하지 않음)
- `{ }` - 중괄호는 함수 본문을 포함

Print 문

```zig
std.debug.print("Hello, World!\n", .{});
```

분석: - `std` - 임포트한 표준 라이브러리 - `.debug` - 디버깅 유틸리티를 위한 std 내의 네임스페이스 - `.print()` - 형식화된 텍스트를 출력하는 함수 - `"Hello, World!\n"` - 형식 문자열 (출력할 텍스트) - `\n` - 새 줄을 위한 이스케이프 시퀀스 - `.{}` - 인자를 위한 빈 익명 구조체 리터럴 (튜플)

변수와 함께 출력하기

변수를 사용하여 프로그램을 더 동적으로 만들어 봅시다:

```zig
const std = @import("std");

pub fn main() void {
    // 문자열 상수 선언
    // "Zig" 는 자동으로 바이트 포인터로 타입이 지정됩니다
    const name = "Zig";

    // 정수 상수 선언
    // : u32 는 명시적으로 부호 없는 32 비트로 타입을 지정합니다
    const year: u32 = 2016;

    // 형식 지정자로 출력:
    // {s} – 문자열로 형식화
    // {d} – 십진수로 형식화
    std.debug.print("{s} 에 오신 것을 환영합니다!\n", .{name});
    std.debug.print("{s} 는 {d} 년에 만들어졌습니다.\n", .{ name, year });
```

```zig
    // 튜플에 여러 값
    const version = "0.11";
    const is_stable = true;
    std.debug.print(" 버전: {s}, 안정판: {}\n", .{ version, is_stable });
}
```

출력:

```
Zig에 오신 것을 환영합니다!
Zig는 2016년에 만들어졌습니다.
버전: 0.11, 안정판: true
```

형식 지정자 참조

형식 지정자	
지정자	설명
{s}	문자열
{d}	십진수 (정수)
{x}	16진수 (소문자)
{X}	16진수 (대문자)
{b}	2진수
{c}	문자
{}	기본 형식 (자동 감지)
{any}	모든 타입 출력 (디버깅용)
{?}	옵셔널 값
{e}	과학적 표기법
수정자	
{d:5}	최소 너비 5
{d:0>5}	오른쪽 정렬, 0으로 채움, 너비 5
{d:<5}	왼쪽 정렬, 너비 5
{d:.2}	소수점 2자리 (실수용)

핵심 포인트

1. 모든 **Zig** 프로그램은 **`main`** 함수가 필요합니다 - 실행이 시작되는 곳입니다
2. 사용 전 임포트 - 라이브러리를 가져오려면 `@import` 사용
3. 형식 문자열은 타입 검사됨 - Zig 는 컴파일 시간에 형식 지정자가 인자와 일치하는지 확인합니다
4. 세미콜론 필수 - JavaScript 와 달리 생략할 수 없습니다
5. 주석은 `//` 사용 - 줄에서 `//` 이후는 무시됩니다

---

2 장: 변수

변수는 값을 저장하는 컨테이너입니다. Zig 는 변경할 수 있는 값과 변경할 수 없는 값을 명확하게 구분합니다.

두 가지 변수 유형

```
┌──────────────────────────────────────────────────────────┐
│                      Zig의 변수 유형                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────────┐   ┌──────────────────────┐    │
│   │       const          │   │        var           │    │
│   │     (불변)           │   │      (가변)          │    │
│   ├──────────────────────┤   ├──────────────────────┤    │
│   │                      │   │                      │    │
│   │  const x = 5;        │   │  var x = 5;          │    │
│   │  x = 10; // 오류!    │   │  x = 10; // OK!      │    │
│   │                      │   │                      │    │
│   │  • 변경 불가         │   │  • 변경 가능         │    │
│   │  • 선호됨            │   │  • 필요할 때 사용    │    │
│   │  • 스레드 안전       │   │  • 주의해서 사용     │    │
│   │  • 최적화 가능       │   │                      │    │
│   └──────────────────────┘   └──────────────────────┘    │
│                                                          │
│   규칙: 수정이 필요하지 않으면 항상 const를 사용하세요!    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

상수 (const)

상수는 초기화 후 절대 변경되지 않는 값입니다. Zig 에서 기본 선택입니다.

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 상수 - 절대 변경되지 않는 값
    // ========================================

    // 타입이 추론됨 (Zig 가 알아냄)
    const greeting = "Hello";           // []const u8 (문자열)
    const answer = 42;                   // comptime_int
    const pi = 3.14159;                  // comptime_float

    // 타입이 명시적 (직접 지정)
    const age: u32 = 25;                 // 부호 없는 32 비트 정수
    const temperature: f64 = 98.6;       // 64 비트 부동소수점
    const is_valid: bool = true;         // 불리언

    std.debug.print("greeting: {s}\n", .{greeting});
    std.debug.print("answer: {d}\n", .{answer});
    std.debug.print("pi: {d:.5}\n", .{pi});
    std.debug.print("age: {d}\n", .{age});
    std.debug.print("temperature: {d}\n", .{temperature});
    std.debug.print("is_valid: {}\n", .{is_valid});

    // 이것은 컴파일 오류를 발생시킵니다:
```

```
    // age = 26;  // 오류: 상수에 할당할 수 없음
}
```

출력:

```
greeting: Hello
answer: 42
pi: 3.14159
age: 25
temperature: 98.6
is_valid: true
```

변수 (var)

변수는 초기화 후 변경할 수 있습니다. 필요할 때만 사용하세요.

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 변수 - 변경할 수 있는 값
    // ========================================

    var counter: i32 = 0;
    std.debug.print(" 초기 카운터: {d}\n", .{counter});

    // 변수 수정
    counter = counter + 1;  // 이제 1
    std.debug.print("+1 후: {d}\n", .{counter});

    counter += 10;          // counter = counter + 10 의 축약형
    std.debug.print("+=10 후: {d}\n", .{counter});

    counter *= 2;           // 두 배로
    std.debug.print("*=2 후: {d}\n", .{counter});

    // var 로 선언할 때 변수는 타입이 있어야 합니다
    var name: []const u8 = "Alice";
    std.debug.print(" 이름: {s}\n", .{name});

    name = "Bob";           // 값 변경
    std.debug.print(" 새 이름: {s}\n", .{name});
}
```

출력:

```
초기 카운터: 0
+1 후: 1
+=10 후: 11
*=2 후: 22
이름: Alice
새 이름: Bob
```

변수의 메모리 레이아웃

```
┌──────────────────────────────────────────────────────────────┐
│                        메모리 레이아웃                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   다음과 같이 작성할 때: var x: i32 = 42;                     │
│                                                              │
│   스택 메모리:                                                │
│   ┌──────────────┬──────────┬──────────────┐                │
│   │  주소        │  값      │  변수        │                │
│   ├──────────────┼──────────┼──────────────┤                │
│   │  0x1000      │   42     │     x        │ ← 4 바이트 (i32) │
│   │  0x1004      │   ...    │   (다음)     │                │
│   └──────────────┴──────────┴──────────────┘                │
│                                                              │
│   다른 타입은 다른 양의 메모리를 사용합니다:                   │
│   ┌──────────┬──────────┬──────────────────────────┐        │
│   │ 타입     │ 크기     │ 메모리 시각화            │        │
│   ├──────────┼──────────┼──────────────────────────┤        │
│   │ u8       │ 1 바이트 │ [  ]                     │        │
│   │ u16      │ 2 바이트 │ [    ]                   │        │
│   │ u32      │ 4 바이트 │ [        ]               │        │
│   │ u64      │ 8 바이트 │ [                ]       │        │
│   │ bool     │ 1 바이트 │ [  ]                     │        │
│   └──────────┴──────────┴──────────────────────────┘        │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Undefined 값

변수를 선언하고 나중에 값을 할당하고 싶을 때 `undefined` 를 사용합니다:

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // UNDEFINED - 지금 선언하고 나중에 할당
    // ========================================

    // 초기화 없이 선언
    // 경고: 할당 전에 undefined 값을 사용하면 정의되지 않은 동작 (UB) 입니다!
    var result: i32 = undefined;

    // ... 값을 결정하는 코드 ...

    // 이제 값 할당
    result = calculateSomething();

    std.debug.print(" 결과: {d}\n", .{result});
}
```

```zig
fn calculateSomething() i32 {
    return 42;
}
```

출력:

```
결과: 42
```

핵심 포인트

1. **var** 보다 **const** 선호 - 불변성이 버그를 방지합니다
2. 타입은 추론되거나 명시적일 수 있음 - Zig 는 가능하면 타입을 알아냅니다
3. **undefined** 는 위험함 - 읽기 전에 할당할 때만 사용
4. 블록 표현식 - 복잡한 초기화에 레이블이 있는 블록 사용
5. 컴파일 시간 평가 - 최상위 상수는 런타임 전에 계산됩니다

---

3 장: 타입

Zig 는 정적 타입 언어입니다. 즉, 모든 값은 컴파일 시간에 알려진 타입을 가집니다. 타입을 이해하는 것은 올바른 Zig 코드를 작성하는 데 기본입니다.

타입 시스템 개요

```
┌──────────────────────────────────────────────────────────┐
│                    Zig 타입 계층 구조                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│                        모든 타입                         │
│                           │                              │
│              ┌────────────┴────────────┐                 │
│              │                         │                 │
│        기본 타입                      복합 타입          │
│              │                         │                 │
│     ┌────────┴────────┐       ┌────────┴────────┐        │
│     │                 │       │                 │        │
│    숫자              기타     배열      구조체     유니온  │
│     │                 │       │         │         │      │
│  ┌──┴──┐          ┌───┴───┐  슬라이스  열거형    옵셔널   │
│  │     │          │       │  포인터                      │
│ 정수  부동소수점 불리언   Void   벡터                      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

정수 타입

Zig 는 정수 크기에 대한 세밀한 제어를 제공합니다:

```
┌──────────────────────────────────────────────────────────┐
│                        정수 타입                         │
├──────────────────────────────────────────────────────────┤
```

```
│                                                                │
│   부호 있음 (음수 가능)            부호 없음 (양수만)          │
│   ────────────────────────       ──────────────────────────   │
│   i8   : -128 ~ 127              u8   : 0 ~ 255               │
│   i16  : -32768 ~ 32767          u16  : 0 ~ 65535             │
│   i32  : -21억 ~ 21억            u32  : 0 ~ 42억              │
│   i64  : 거대한 범위             u64  : 0 ~ 거대              │
│   i128 : 더 거대                 u128 : 더 거대               │
│                                                                │
│   특수 타입:                                                   │
│   ────────────────                                             │
│   isize : 부호 있는 포인터 크기 (32 또는 64비트)               │
│   usize : 부호 없는 포인터 크기 (배열 인덱스용)                │
│                                                                │
│   임의 너비:                                                   │
│   ──────────────────                                           │
│   u1, u2, u3, ..., u65535  (어떤 비트 너비든!)                 │
│   i1, i2, i3, ..., i65535                                      │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 부호 있는 정수 (음수 가능)
    // ========================================

    const tiny: i8 = -128;        // 8 비트: -128 ~ 127
    const small: i16 = -32768;    // 16 비트
    const medium: i32 = -2147483648; // 32 비트
    const large: i64 = -9223372036854775808; // 64 비트

    std.debug.print("i8 최소:  {d}\n", .{tiny});
    std.debug.print("i16 최소: {d}\n", .{small});
    std.debug.print("i32 최소: {d}\n", .{medium});
    std.debug.print("i64 최소: {d}\n", .{large});

    // ========================================
    // 부호 없는 정수 (양수만)
    // ========================================

    const byte: u8 = 255;          // 8 비트: 0 ~ 255
    const word: u16 = 65535;       // 16 비트
    const dword: u32 = 4294967295; // 32 비트
    const qword: u64 = 18446744073709551615; // 64 비트

    std.debug.print("\nu8 최대:  {d}\n", .{byte});
    std.debug.print("u16 최대: {d}\n", .{word});
    std.debug.print("u32 최대: {d}\n", .{dword});
    std.debug.print("u64 최대: {d}\n", .{qword});

    // ========================================
```

```zig
    // 임의 비트 너비
    // ========================================

    const nibble: u4 = 15;          // 4 비트: 0 ~ 15
    const six_bits: u6 = 63;        // 6 비트: 0 ~ 63
    const twelve: u12 = 4095;       // 12 비트: 0 ~ 4095

    std.debug.print("\nu4 최대:  {d}\n", .{nibble});
    std.debug.print("u6 최대:  {d}\n", .{six_bits});
    std.debug.print("u12 최대: {d}\n", .{twelve});
}
```

출력:

```
i8 최소:  -128
i16 최소: -32768
i32 최소: -2147483648
i64 최소: -9223372036854775808

u8 최대:  255
u16 최대: 65535
u32 최대: 4294967295
u64 최대: 18446744073709551615

u4 최대:  15
u6 최대:  63
u12 최대: 4095
```

부동소수점 타입

```
┌──────────────────────────────────────────────────────────┐
│                      부동소수점 타입                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   타입     크기       정밀도         범위 (대략)          │
│   ────     ────     ──────────     ──────────────        │
│   f16      16비트    ~3자리         ±65504               │
│   f32      32비트    ~7자리         ±3.4 × 10^38         │
│   f64      64비트    ~15자리        ±1.8 × 10^308        │
│   f128     128비트   ~33자리        ±1.2 × 10^4932       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 부동소수점 숫자
    // ========================================

    const pi_32: f32 = 3.14159265358979;  // 덜 정밀
    const pi_64: f64 = 3.14159265358979;  // 더 정밀
```

```zig
    std.debug.print("f32 pi: {d:.10}\n", .{pi_32});
    std.debug.print("f64 pi: {d:.14}\n", .{pi_64});

    // 과학적 표기법
    const avogadro: f64 = 6.022e23;  // 6.022 × 10^23
    const planck: f64 = 6.626e-34;   // 6.626 × 10^-34

    std.debug.print(" 아보가드로: {e}\n", .{avogadro});
    std.debug.print(" 플랑크: {e}\n", .{planck});
}
```

출력:

```
f32 pi: 3.1415927410
f64 pi: 3.14159265358979
아보가드로: 6.022e23
플랑크: 6.626e-34
```

불리언 타입

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 불리언 - true 또는 false
    // ========================================

    const is_active: bool = true;
    const is_empty: bool = false;

    std.debug.print("is_active: {}\n", .{is_active});
    std.debug.print("is_empty: {}\n", .{is_empty});

    // 비교에서 불리언
    const x: i32 = 10;
    const y: i32 = 20;

    const is_equal = x == y;     // false
    const is_less = x < y;       // true
    const is_greater = x > y;    // false

    std.debug.print("\n{d} == {d}: {}\n", .{ x, y, is_equal });
    std.debug.print("{d} < {d}: {}\n", .{ x, y, is_less });
    std.debug.print("{d} > {d}: {}\n", .{ x, y, is_greater });

    // 메모리에서 불리언은 1 바이트
    std.debug.print("\nbool 크기: {d} 바이트\n", .{@sizeOf(bool)});
}
```

출력:

```
is_active: true
is_empty: false
```

```
10 == 20: false
10 < 20: true
10 > 20: false

bool 크기: 1 바이트
```

핵심 포인트

1. 정수는 크기와 부호가 명시적 - i32, u64 등
2. 임의 비트 너비 가능 - u4, i12 등 어떤 크기든 가능
3. 부동소수점은 정밀도 주의 - 이진 표현으로 인한 오차
4. 타입 캐스팅은 명시적 - @intCast, @floatCast 사용

---

4 장: 연산자

Zig 의 연산자는 명시적이고 예측 가능하도록 설계되었습니다.

산술 연산자

```
┌──────────────────────────────────────────────────────────┐
│                        산술 연산자                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   연산자     설명              예제                      │
│   ──────     ────              ────                      │
│    +         덧셈              5 + 3 = 8                 │
│    -         뺄셈              5 - 3 = 2                 │
│    *         곱셈              5 * 3 = 15                │
│    /         나눗셈            6 / 3 = 2                 │
│    %         나머지            7 % 3 = 1                 │
│                                                          │
│   오버플로우 검사:                                        │
│   ──────────────────                                     │
│   +%  -%  *%     래핑 (오버플로우 허용)                   │
│   +|  -|  *|     포화 (최대/최소에서 멈춤)               │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // 기본 산술
    const a: i32 = 10;
    const b: i32 = 3;

    std.debug.print("a + b = {d}\n", .{a + b});  // 13
    std.debug.print("a - b = {d}\n", .{a - b});  // 7
    std.debug.print("a * b = {d}\n", .{a * b});  // 30
    std.debug.print("a / b = {d}\n", .{@divTrunc(a, b)});  // 3
    std.debug.print("a % b = {d}\n", .{@mod(a, b)});  // 1
```

```zig
    // 오버플로우 검사 산술
    const max: u8 = 255;
    const wrapped = max +% 1;  // 0 으로 래핑
    std.debug.print("\n255 +% 1 = {d} (래핑)\n", .{wrapped});

    const saturated = max +| 1;  // 255 에서 멈춤
    std.debug.print("255 +| 1 = {d} (포화)\n", .{saturated});
}
```

출력:

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1

255 +% 1 = 0 (래핑)
255 +| 1 = 255 (포화)
```

비교 연산자

```zig
const std = @import("std");

pub fn main() void {
    const x: i32 = 10;
    const y: i32 = 20;

    std.debug.print("x == y: {}\n", .{x == y});  // false
    std.debug.print("x != y: {}\n", .{x != y});  // true
    std.debug.print("x < y: {}\n", .{x < y});    // true
    std.debug.print("x <= y: {}\n", .{x <= y});  // true
    std.debug.print("x > y: {}\n", .{x > y});    // false
    std.debug.print("x >= y: {}\n", .{x >= y});  // false
}
```

출력:

```
x == y: false
x != y: true
x < y: true
x <= y: true
x > y: false
x >= y: false
```

논리 연산자

```zig
const std = @import("std");

pub fn main() void {
    const a = true;
    const b = false;
```

```
    std.debug.print("a and b: {}\n", .{a and b});  // false
    std.debug.print("a or b: {}\n", .{a or b});    // true
    std.debug.print("not a: {}\n", .{!a});         // false
}
```

출력:

```
a and b: false
a or b: true
not a: false
```

비트 연산자

```
const std = @import("std");

pub fn main() void {
    const a: u8 = 0b11110000;  // 240
    const b: u8 = 0b10101010;  // 170

    std.debug.print("a & b = {b:0>8}\n", .{a & b});   // AND
    std.debug.print("a | b = {b:0>8}\n", .{a | b});   // OR
    std.debug.print("a ^ b = {b:0>8}\n", .{a ^ b});   // XOR
    std.debug.print("~a = {b:0>8}\n", .{~a});         // NOT

    // 시프트
    const x: u8 = 0b00001111;
    std.debug.print("\nx = {b:0>8}\n", .{x});
    std.debug.print("x << 2 = {b:0>8}\n", .{x << 2});  // 왼쪽 시프트
    std.debug.print("x >> 2 = {b:0>8}\n", .{x >> 2});  // 오른쪽 시프트
}
```

출력:

```
a & b = 10100000
a | b = 11111010
a ^ b = 01011010
~a = 00001111

x = 00001111
x << 2 = 00111100
x >> 2 = 00000011
```

핵심 포인트

1. 오버플로우 검사 - 기본 연산은 오버플로우를 검사합니다
2. 래핑/포화 연산 - +%, +| 로 오버플로우 동작 제어
3. 논리 연산은 **and/or** - &&, || 가 아닙니다
4. 비트 연산 - 저수준 비트 조작에 사용

---

5 장: 배열

배열은 동일한 타입의 고정 크기 요소 컬렉션입니다.

배열 메모리 레이아웃

```
┌──────────────────────────────────────────────────────────┐
│                      배열 메모리 레이아웃                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   const arr: [5]i32 = [5]i32{10, 20, 30, 40, 50};        │
│                                                          │
│   메모리:                                                 │
│   ┌────────┬────────┬────────┬────────┬────────┐         │
│   │   10   │   20   │   30   │   40   │   50   │         │
│   └────────┴────────┴────────┴────────┴────────┘         │
│   [0]     [1]     [2]     [3]     [4]                    │
│                                                          │
│   • 연속 메모리                                           │
│   • 인덱스 0부터 시작                                     │
│   • 크기 컴파일 시간에 알려짐                              │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // 배열 선언
    const numbers = [5]i32{ 10, 20, 30, 40, 50 };

    // 인덱스로 접근
    std.debug.print(" 첫 번째: {d}\n", .{numbers[0]});  // 10
    std.debug.print(" 마지막: {d}\n", .{numbers[4]});   // 50

    // 전체 배열 출력
    std.debug.print(" 배열: {any}\n", .{numbers});

    // 배열 반복
    std.debug.print("\n반복:\n", .{});
    for (numbers, 0..) |num, idx| {
        std.debug.print("  [{d}] = {d}\n", .{ idx, num });
    }

    // 가변 배열
    var mutable = [_]i32{ 1, 2, 3 };
    mutable[0] = 100;
    std.debug.print("\n수정된 배열: {any}\n", .{mutable});

    // 배열 길이
    std.debug.print(" 길이: {d}\n", .{numbers.len});
}
```

출력:

```
첫 번째: 10
마지막: 50
배열: { 10, 20, 30, 40, 50 }
```

```
반복:
  [0] = 10
  [1] = 20
  [2] = 30
  [3] = 40
  [4] = 50

수정된 배열: { 100, 2, 3 }
길이: 5
```

다차원 배열

```zig
const std = @import("std");

pub fn main() void {
    // 2 차원 배열 (행렬)
    const matrix = [3][3]i32{
        [_]i32{ 1, 2, 3 },
        [_]i32{ 4, 5, 6 },
        [_]i32{ 7, 8, 9 },
    };

    std.debug.print(" 행렬:\n", .{});
    for (matrix) |row| {
        for (row) |val| {
            std.debug.print("{d} ", .{val});
        }
        std.debug.print("\n", .{});
    }

    std.debug.print("\n중앙 값: {d}\n", .{matrix[1][1]});  // 5
}
```

출력:

```
행렬:
1 2 3
4 5 6
7 8 9

중앙 값: 5
```

핵심 포인트

1. 고정 크기 - 배열 크기는 컴파일 시간에 알려져야 함
2. 동일 타입 - 모든 요소는 같은 타입
3. 연속 메모리 - 요소가 메모리에 연속으로 저장됨
4. 경계 검사 - 범위를 벗어난 접근은 오류 발생

---

6 장: 슬라이스

슬라이스는 배열의 연속된 부분에 대한 뷰입니다.

슬라이스 vs 배열

```
┌──────────────────────────────────────────────────────────────┐
│                        배열 vs 슬라이스                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   배열: [5]i32                                               │
│   ┌────┬────┬────┬────┬────┐                                │
│   │ 10 │ 20 │ 30 │ 40 │ 50 │                                │
│   └────┴────┴────┴────┴────┘                                │
│   • 크기가 타입의 일부                                        │
│   • 스택에 데이터 저장                                        │
│                                                              │
│   슬라이스: []i32                                            │
│   ┌──────┬──────┐                                            │
│   │ ptr  │ len  │ ──────► [  30  |  40  |  50  ]             │
│   └──────┴──────┘            ↑                               │
│   • 포인터 + 길이                                             │
│   • 배열의 뷰 (부분)                                          │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // 배열에서 슬라이스 생성
    const arr = [_]i32{ 10, 20, 30, 40, 50 };
    const slice = arr[1..4];  // 인덱스 1, 2, 3

    std.debug.print(" 원본 배열: {any}\n", .{arr});
    std.debug.print(" 슬라이스 [1..4]: {any}\n", .{slice});
    std.debug.print(" 슬라이스 길이: {d}\n", .{slice.len});

    // 다양한 슬라이스 범위
    std.debug.print("\narr[0..2]: {any}\n", .{arr[0..2]});  // 처음 2 개
    std.debug.print("arr[2..]: {any}\n", .{arr[2..]});      // 인덱스 2 부터 끝까지
    std.debug.print("arr[..3]: {any}\n", .{arr[0..3]});     // 처음부터 인덱스 3 전까지

    // 슬라이스 반복
    std.debug.print("\n슬라이스 반복:\n", .{});
    for (slice, 0..) |val, idx| {
        std.debug.print("  [{d}] = {d}\n", .{ idx, val });
    }
}
```

출력:

```
원본 배열: { 10, 20, 30, 40, 50 }
슬라이스 [1..4]: { 20, 30, 40 }
슬라이스 길이: 3

arr[0..2]: { 10, 20 }
arr[2..]: { 30, 40, 50 }
arr[..3]: { 10, 20, 30 }
```

```
슬라이스 반복:
  [0] = 20
  [1] = 30
  [2] = 40
```

가변 슬라이스

```zig
const std = @import("std");

pub fn main() void {
    var arr = [_]i32{ 1, 2, 3, 4, 5 };
    var slice = arr[1..4];

    std.debug.print(" 수정 전: {any}\n", .{slice});

    // 슬라이스를 통해 원본 수정
    slice[0] = 100;
    slice[1] = 200;

    std.debug.print(" 수정 후: {any}\n", .{slice});
    std.debug.print(" 원본 배열: {any}\n", .{arr});
}
```

출력:

```
수정 전: { 2, 3, 4 }
수정 후: { 100, 200, 4 }
원본 배열: { 1, 100, 200, 4, 5 }
```

핵심 포인트

1. 포인터 + 길이 - 슬라이스는 시작 위치와 길이로 구성
2. 원본 참조 - 슬라이스 수정은 원본에 영향
3. 동적 길이 - 런타임에 길이가 결정될 수 있음
4. 경계 검사 - 안전한 범위 접근

---

7 장: 포인터

포인터는 메모리 주소를 저장합니다. Zig 의 포인터는 C 보다 더 안전합니다.

포인터 개념

```
┌──────────────────────────────────────────────────────────┐
│                        포인터 개념                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   var x: i32 = 42;                                       │
│   const ptr: *i32 = &x;                                  │
│                                                          │
│   ┌──────────────┐       ┌──────────────┐               │
```

```
│   │ ptr          │       │ x            │                    │
│   │ ────────────  │──────│ ────────────  │                    │
│   │  0x1000      │──────→│     42       │                    │
│   └──────────────┘       └──────────────┘                    │
│    주소 저장              실제 값                             │
│                                                               │
│   ptr.*  = 42  (역참조: 값 읽기)                              │
│   &x     = ptr (주소 가져오기)                                │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    var x: i32 = 42;
    const ptr: *i32 = &x;  // x 의 주소

    std.debug.print("x 의 값: {d}\n", .{x});
    std.debug.print(" 포인터가 가리키는 값: {d}\n", .{ptr.*});

    // 포인터를 통해 값 수정
    ptr.* = 100;
    std.debug.print(" 수정 후 x: {d}\n", .{x});

    // const 포인터 vs 일반 포인터
    const y: i32 = 10;
    const const_ptr: *const i32 = &y;  // 값 수정 불가
    std.debug.print("const 포인터 값: {d}\n", .{const_ptr.*});
}
```

출력:

```
x의 값: 42
포인터가 가리키는 값: 42
수정 후 x: 100
const 포인터 값: 10
```

포인터 타입

```
┌──────────────────────────────────────────────────────────────┐
│                        포인터 타입                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   *T            단일 항목 포인터 (null 불가)                 │
│   *const T      읽기 전용 단일 항목 포인터                   │
│   [*]T          다중 항목 포인터 (길이 없음)                 │
│   ?*T           옵셔널 포인터 (null 가능)                    │
│   [*c]T         C 호환 포인터                                │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

핵심 포인트

1. **null** 안전성 - 기본 포인터는 null 이 될 수 없음
2. 역참조 - `ptr.*` 로 값에 접근
3. 주소 가져오기 - `&variable` 로 주소 얻음
4. 타입 구분 - const 와 mutable 포인터 구분

---

8 장: 문자열

Zig 에서 문자열은 바이트의 시퀀스입니다.

문자열 타입

```
┌──────────────────────────────────────────────────────────┐
│                      문자열 타입                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   []const u8        바이트 슬라이스 (일반 문자열)         │
│   [:0]const u8      null 종료 문자열                      │
│   [N]u8             고정 크기 바이트 배열                 │
│   *const [N]u8      바이트 배열 포인터                    │
│                                                          │
│   "hello"의 메모리:                                      │
│   ┌───┬───┬───┬───┬───┬───┐                             │
│   │ h │ e │ l │ l │ o │ 0 │                             │
│   └───┴───┴───┴───┴───┴───┘                             │
│   104 101 108 108 111  0   (ASCII 값)                    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // 문자열 리터럴
    const greeting: []const u8 = "Hello, Zig!";

    std.debug.print(" 문자열: {s}\n", .{greeting});
    std.debug.print(" 길이: {d}\n", .{greeting.len});

    // 문자 접근
    std.debug.print(" 첫 번째 문자: {c}\n", .{greeting[0]});  // 'H'

    // 문자열 반복
    std.debug.print("\n문자별 출력:\n", .{});
    for (greeting) |char| {
        std.debug.print("{c} ", .{char});
    }
    std.debug.print("\n", .{});

    // 문자열 슬라이싱
    const hello = greeting[0..5];
```

```zig
    std.debug.print("\n처음 5 자: {s}\n", .{hello});

    // 멀티라인 문자열
    const multiline =
        \\첫 번째 줄
        \\두 번째 줄
        \\세 번째 줄
    ;
    std.debug.print("\n멀티라인:\n{s}\n", .{multiline});
}
```

출력:

```
문자열: Hello, Zig!
길이: 11
첫 번째 문자: H

문자별 출력:
H e l l o ,   Z i g !

처음 5자: Hello

멀티라인:
첫 번째 줄
두 번째 줄
세 번째 줄
```

핵심 포인트

1. **UTF-8** - Zig 문자열은 UTF-8 인코딩
2. 바이트 기반 - 문자열은 u8 의 슬라이스
3. 불변 - 리터럴 문자열은 수정 불가
4. **null** 종료 - C 호환을 위한 `[:0]` 타입 제공

---

9 장: 구조체

구조체는 관련 데이터를 그룹화하는 복합 타입입니다.

구조체 메모리 레이아웃

```
┌──────────────────────────────────────────────────────────┐
│                    구조체 메모리 레이아웃                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   const Person = struct {                                │
│       name: []const u8,                                  │
│       age: u32,                                          │
│       active: bool,                                      │
│   };                                                     │
│                                                          │
```

```
│   메모리:                                                    │
│   ┌──────────────┬──────────┬──────────┐                    │
│   │  name (ptr)  │   age    │ active   │                    │
│   │   8 bytes    │ 4 bytes  │ 1 byte   │ + 패딩             │
│   └──────────────┴──────────┴──────────┘                    │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 구조체 정의
const Person = struct {
    name: []const u8,
    age: u32,
    active: bool = true,  // 기본값

    // 메서드
    pub fn introduce(self: Person) void {
        std.debug.print(" 안녕하세요, 저는 {s} 이고 {d} 살입니다.\n",
            .{ self.name, self.age });
    }

    pub fn birthday(self: *Person) void {
        self.age += 1;
    }
};

pub fn main() void {
    // 구조체 인스턴스 생성
    var person = Person{
        .name = " 철수",
        .age = 25,
    };

    person.introduce();

    // 필드 수정
    person.birthday();
    std.debug.print(" 생일 후: {d} 살\n", .{person.age});

    // 익명 구조체
    const point = .{
        .x = 10,
        .y = 20,
    };
    std.debug.print("\n점: ({d}, {d})\n", .{ point.x, point.y });

    // 구조체 크기
    std.debug.print("\nPerson 크기: {d} 바이트\n", .{@sizeOf(Person)});
}
```

출력:

```
안녕하세요, 저는 철수이고 25살입니다.
생일 후: 26살
```

```
점: (10, 20)

Person 크기: 24 바이트
```

핵심 포인트

1. 관련 데이터 그룹화 - 여러 필드를 하나의 타입으로
2. 메서드 지원 - 구조체에 함수 연결 가능
3. 기본값 - 필드에 기본값 지정 가능
4. 익명 구조체 - 타입 없이 즉석에서 생성

---

10 장: 열거형

열거형은 명명된 상수들의 집합입니다.

열거형 개념

```
┌──────────────────────────────────────────────────────────┐
│                        열거형 개념                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   const Color = enum { red, green, blue };               │
│                                                          │
│   ┌──────────────┬──────────────┬──────────────┐         │
│   │     red      │     green    │     blue     │         │
│   │      0       │      1       │      2       │         │
│   └──────────────┴──────────────┴──────────────┘         │
│                                                          │
│   • 타입 안전한 상수                                      │
│   • 자동으로 0부터 번호 매김                              │
│   • switch와 완벽하게 작동                                │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 열거형 정의
const Color = enum {
    red,
    green,
    blue,

    // 메서드
    pub fn toRgb(self: Color) struct { r: u8, g: u8, b: u8 } {
        return switch (self) {
            .red => .{ .r = 255, .g = 0, .b = 0 },
            .green => .{ .r = 0, .g = 255, .b = 0 },
            .blue => .{ .r = 0, .g = 0, .b = 255 },
        };
```

```zig
    }
};

// 명시적 값이 있는 열거형
const Status = enum(u8) {
    pending = 1,
    active = 2,
    completed = 10,
};

pub fn main() void {
    const color = Color.green;

    std.debug.print(" 색상: {}\n", .{color});

    // switch 와 함께 사용
    const name = switch (color) {
        .red => " 빨강",
        .green => " 초록",
        .blue => " 파랑",
    };
    std.debug.print(" 이름: {s}\n", .{name});

    // 메서드 호출
    const rgb = color.toRgb();
    std.debug.print("RGB: ({d}, {d}, {d})\n", .{ rgb.r, rgb.g, rgb.b });

    // 정수로 변환
    const status = Status.completed;
    std.debug.print("\n상태 값: {d}\n", .{@intFromEnum(status)});
}
```

출력:

```
색상: enum.green
이름: 초록
RGB: (0, 255, 0)

상태 값: 10
```

핵심 포인트

1. 타입 안전 - 잘못된 값 할당 방지
2. **switch** 완전성 - 모든 케이스 처리 강제
3. 메서드 가능 - 열거형에 함수 연결
4. 명시적 값 - 필요시 정수 값 지정

---

11 장: 유니온

유니온은 같은 메모리 위치에 다른 타입의 값을 저장할 수 있습니다.

태그된 유니온

```
┌──────────────────────────────────────────────────────────────┐
│                        태그된 유니온                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   const Value = union(enum) {                                │
│       int: i32,                                              │
│       float: f64,                                            │
│       none,                                                  │
│   };                                                         │
│                                                              │
│   메모리 레이아웃:                                              │
│   ┌──────────┬──────────────────────────┐                   │
│   │   태그    │          페이로드          │                   │
│   │   (enum) │   (가장 큰 타입 크기)       │                   │
│   └──────────┴──────────────────────────┘                   │
│                                                              │
│   한 번에 하나의 필드만 활성화됨                                │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 태그된 유니온
const Value = union(enum) {
    int: i32,
    float: f64,
    boolean: bool,
    none,

    pub fn format(self: Value) void {
        switch (self) {
            .int => |i| std.debug.print(" 정수: {d}\n", .{i}),
            .float => |f| std.debug.print(" 실수: {d:.2}\n", .{f}),
            .boolean => |b| std.debug.print(" 불리언: {}\n", .{b}),
            .none => std.debug.print(" 없음\n", .{}),
        }
    }
};

pub fn main() void {
    // 다른 타입의 값 저장
    var val = Value{ .int = 42 };
    val.format();

    val = Value{ .float = 3.14 };
    val.format();

    val = Value{ .boolean = true };
    val.format();

    val = Value.none;
```

```
    val.format();

    // 유니온 크기
    std.debug.print("\nValue 크기: {d} 바이트\n", .{@sizeOf(Value)});
}
```

출력:

```
정수: 42
실수: 3.14
불리언: true
없음

Value 크기: 16 바이트
```

핵심 포인트

1. 메모리 효율 - 모든 필드가 같은 메모리 공유
2. 태그된 유니온 - 현재 활성 필드 추적
3. **switch** 와 함께 - 안전한 값 접근
4. 하나의 필드만 - 동시에 하나의 값만 저장

---

12 장: 제어 흐름

Zig 의 제어 흐름 구조는 명확하고 명시적입니다.

if 표현식

```zig
const std = @import("std");

pub fn main() void {
    const x: i32 = 10;

    // 기본 if
    if (x > 5) {
        std.debug.print("x 는 5 보다 큽니다\n", .{});
    }

    // if-else
    if (x > 20) {
        std.debug.print("x 는 20 보다 큽니다\n", .{});
    } else if (x > 10) {
        std.debug.print("x 는 10 보다 큽니다\n", .{});
    } else {
        std.debug.print("x 는 10 이하입니다\n", .{});
    }

    // if 표현식 (값 반환)
    const message = if (x > 0) " 양수" else " 음수 또는 0";
    std.debug.print("x 는 {s} 입니다\n", .{message});
```

```zig
    // 옵셔널과 if
    const maybe: ?i32 = 42;
    if (maybe) |value| {
        std.debug.print(" 값: {d}\n", .{value});
    }
}
```

출력:

```
x는 5보다 큽니다
x는 10 이하입니다
x는 양수입니다
값: 42
```

switch

```
┌──────────────────────────────────────────────────────────┐
│                      switch 기능                         │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   • 완전성 검사 - 모든 경우 처리 필수                    │
│   • 범위 지원 - 5...10                                   │
│   • 다중 값 - 1, 2, 3 => ...                             │
│   • else 분기 - 기타 모든 경우                           │
│   • 값 반환 가능                                         │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    const value: u8 = 5;

    // 기본 switch
    const result = switch (value) {
        0 => " 영",
        1, 2, 3 => " 작은 수",
        4...10 => " 중간 수",
        else => " 큰 수",
    };
    std.debug.print(" 결과: {s}\n", .{result});

    // switch 로 코드 실행
    const day: u8 = 3;
    switch (day) {
        1 => std.debug.print(" 월요일\n", .{}),
        2 => std.debug.print(" 화요일\n", .{}),
        3 => std.debug.print(" 수요일\n", .{}),
        else => std.debug.print(" 다른 요일\n", .{}),
    }
}
```

출력:

```
결과: 중간 수
수요일
```

반복문

```zig
const std = @import("std");

pub fn main() void {
    // while 루프
    var i: u32 = 0;
    while (i < 5) : (i += 1) {
        std.debug.print("while: {d}\n", .{i});
    }

    // for 루프 (배열/슬라이스)
    const arr = [_]i32{ 10, 20, 30 };
    std.debug.print("\nfor 루프:\n", .{});
    for (arr, 0..) |val, idx| {
        std.debug.print("  [{d}] = {d}\n", .{ idx, val });
    }

    // for 루프 (범위)
    std.debug.print("\n범위 0..5:\n", .{});
    for (0..5) |n| {
        std.debug.print("{d} ", .{n});
    }
    std.debug.print("\n", .{});
}
```

출력:

```
while: 0
while: 1
while: 2
while: 3
while: 4

for 루프:
  [0] = 10
  [1] = 20
  [2] = 30

범위 0..5:
0 1 2 3 4
```

핵심 포인트

1. **if** 는 표현식 - 값을 반환할 수 있음
2. **switch** 완전성 - 모든 경우 처리 필수
3. **for** 는 이터레이터용 - 배열, 슬라이스, 범위에 사용
4. **while** 은 조건부 - 조건이 참인 동안 반복

---

13 장: 함수

함수는 재사용 가능한 코드 블록입니다.

함수 구조

```
┌──────────────────────────────────────────────────────────┐
│                      함수 구조                            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   pub fn add(a: i32, b: i32) i32 {                       │
│       │    │    │         │    │                        │
│       │    │    │         │    └── 반환 타입             │
│       │    │    │         └── 매개변수 2                 │
│       │    │    └── 매개변수 1                           │
│       │    └── 함수 이름                                 │
│       └── 가시성 (public)                                │
│                                                          │
│       return a + b;                                      │
│   }                                                      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 기본 함수
fn add(a: i32, b: i32) i32 {
    return a + b;
}

// 여러 반환 값 (튜플)
fn divmod(a: i32, b: i32) struct { quot: i32, rem: i32 } {
    return .{
        .quot = @divTrunc(a, b),
        .rem = @mod(a, b),
    };
}

// 오류를 반환할 수 있는 함수
fn safeDivide(a: i32, b: i32) !i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

pub fn main() void {
    // 기본 함수 호출
    const sum = add(5, 3);
    std.debug.print("5 + 3 = {d}\n", .{sum});

    // 다중 반환 값
    const result = divmod(17, 5);
    std.debug.print("17 / 5 = {d} 나머지 {d}\n", .{ result.quot, result.rem });
```

```zig
    // 오류 처리
    if (safeDivide(10, 2)) |val| {
        std.debug.print("10 / 2 = {d}\n", .{val});
    } else |err| {
        std.debug.print(" 오류: {}\n", .{err});
    }

    if (safeDivide(10, 0)) |val| {
        std.debug.print("10 / 0 = {d}\n", .{val});
    } else |err| {
        std.debug.print(" 오류: {}\n", .{err});
    }
}
```

출력:

```
5 + 3 = 8
17 / 5 = 3 나머지 2
10 / 2 = 5
오류: error.DivisionByZero
```

핵심 포인트

1. 명시적 타입 - 매개변수와 반환 타입 필수
2. 다중 반환 - 구조체로 여러 값 반환
3. 오류 반환 - ! 로 오류 가능성 표시
4. **pub** 가시성 - 외부에서 접근 가능하게 하려면 pub 사용

---

14 장: 오류 처리

Zig 는 명시적이고 강력한 오류 처리 메커니즘을 제공합니다.

오류 유니온

```
┌──────────────────────────────────────────────────────────┐
│                      오류 유니온 개념                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   fn divide(a: i32, b: i32) !i32                         │
│                              │                           │
│                              └── 오류 유니온 (error!T)    │
│                                                          │
│   반환값:                                                 │
│   ┌──────────────────────────────────────┐               │
│   │        성공          │        실패     │               │
│   ├──────────────────────────────────────┤               │
│   │    실제 값 (T)       │  error.SomeError │               │
│   │       i32            │                 │               │
│   └──────────────────────────────────────┘               │
│                                                          │
```

```zig
const std = @import("std");

// 오류 집합 정의
const MathError = error{
    DivisionByZero,
    Overflow,
    InvalidInput,
};

// 오류를 반환할 수 있는 함수
fn safeDivide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

fn sqrt(x: i32) MathError!i32 {
    if (x < 0) return error.InvalidInput;
    // 간단한 정수 제곱근 구현
    var result: i32 = 0;
    while (result * result <= x) : (result += 1) {}
    return result - 1;
}

pub fn main() void {
    // try - 오류 전파
    const result = safeDivide(10, 2) catch |err| {
        std.debug.print(" 오류: {}\n", .{err});
        return;
    };
    std.debug.print("10 / 2 = {d}\n", .{result});

    // catch 로 기본값 제공
    const safeResult = safeDivide(10, 0) catch 0;
    std.debug.print(" 오류 시 기본값: {d}\n", .{safeResult});

    // if-else 로 오류 처리
    if (sqrt(-4)) |val| {
        std.debug.print("sqrt = {d}\n", .{val});
    } else |err| {
        std.debug.print("sqrt 오류: {}\n", .{err});
    }

    // catch unreachable - 오류가 없다고 확신할 때
    const definiteResult = safeDivide(10, 5) catch unreachable;
    std.debug.print(" 확실한 결과: {d}\n", .{definiteResult});
}
```

출력:

```
10 / 2 = 5
오류 시 기본값: 0
sqrt 오류: error.InvalidInput
확실한 결과: 2
```

핵심 포인트

1. 명시적 오류 - 오류가 가능한 함수는 ! 사용
2. **try/catch** - 오류 전파 또는 처리
3. 오류 집합 - 관련 오류들을 그룹화
4. 강제 처리 - 오류를 무시할 수 없음

---

15 장: 옵셔널

옵셔널은 값이 있거나 없을 수 있는 타입입니다.

옵셔널 개념

```
┌──────────────────────────────────────────────────────────┐
│                      옵셔널 개념                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   const maybe: ?i32 = ...;                               │
│                                                          │
│   ┌──────────────────────────────────────┐               │
│   │   값이 있음          │     값이 없음      │           │
│   ├──────────────────────────────────────┤               │
│   │      42             │       null       │             │
│   │   (실제 값)          │                  │             │
│   └──────────────────────────────────────┘               │
│                                                          │
│   ?T = T 또는 null                                       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 옵셔널을 반환하는 함수
fn findIndex(haystack: []const u8, needle: u8) ?usize {
    for (haystack, 0..) |char, idx| {
        if (char == needle) return idx;
    }
    return null;
}

pub fn main() void {
    const text = "Hello, World!";

    // 값이 있는 경우
    if (findIndex(text, 'o')) |index| {
        std.debug.print("'o'를 인덱스 {d} 에서 찾았습니다\n", .{index});
    } else {
        std.debug.print("'o'를 찾지 못했습니다\n", .{});
    }

    // 값이 없는 경우
```

```zig
    if (findIndex(text, 'z')) |index| {
        std.debug.print("'z'를 인덱스 {d} 에서 찾았습니다\n", .{index});
    } else {
        std.debug.print("'z'를 찾지 못했습니다\n", .{});
    }

    // orelse 로 기본값 제공
    const idx = findIndex(text, 'z') orelse 999;
    std.debug.print("'z' 인덱스 (기본값 사용): {d}\n", .{idx});

    // .? 로 값 추출 (값이 있다고 확신할 때)
    const definiteIdx = findIndex(text, 'H').?;
    std.debug.print("'H' 인덱스: {d}\n", .{definiteIdx});

    // 옵셔널 포인터
    var value: i32 = 42;
    var maybe_ptr: ?*i32 = &value;

    if (maybe_ptr) |ptr| {
        std.debug.print(" 포인터 값: {d}\n", .{ptr.*});
    }

    maybe_ptr = null;
    std.debug.print(" 포인터가 null 인가: {}\n", .{maybe_ptr == null});
}
```

출력:

```
'o'를 인덱스 4에서 찾았습니다
'z'를 찾지 못했습니다
'z' 인덱스 (기본값 사용): 999
'H' 인덱스: 0
포인터 값: 42
포인터가 null인가: true
```

핵심 포인트

1. **null** 안전성 - null 이 가능한 곳을 명시
2. **if** 로 언래핑 - 값이 있을 때만 사용
3. **orelse** 기본값 - null 일 때 대체값 제공
4. **.?** 강제 언래핑 - 값이 있다고 확신할 때

---

16 장: Comptime

Comptime 은 컴파일 시간에 코드를 실행하는 Zig 의 강력한 기능입니다.

```zig
const std = @import("std");

// 컴파일 시간에 계산되는 함수
fn factorial(comptime n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
```

```zig
}

// 컴파일 시간에 배열 생성
fn makeSquares(comptime n: usize) [n]i32 {
    var result: [n]i32 = undefined;
    for (0..n) |i| {
        result[i] = @intCast(i * i);
    }
    return result;
}

pub fn main() void {
    // 컴파일 시간 상수
    const fact_5 = comptime factorial(5);
    std.debug.print("5! = {d} (컴파일 시간에 계산됨)\n", .{fact_5});

    // 컴파일 시간에 생성된 배열
    const squares = comptime makeSquares(5);
    std.debug.print(" 제곱 수: {any}\n", .{squares});

    // 타입 정보 쿼리
    std.debug.print("\ni32 크기: {d} 바이트\n", .{@sizeOf(i32)});
    std.debug.print("i32 비트 수: {d}\n", .{@bitSizeOf(i32)});

    // 컴파일 시간 분기
    const platform = comptime if (@import("builtin").os.tag == .linux)
        "Linux"
    else if (@import("builtin").os.tag == .macos)
        "macOS"
    else
        " 기타";

    std.debug.print(" 플랫폼: {s}\n", .{platform});
}
```

출력:

```
5! = 120 (컴파일 시간에 계산됨)
제곱 수: { 0, 1, 4, 9, 16 }

i32 크기: 4 바이트
i32 비트 수: 32
플랫폼: macOS
```

핵심 포인트

1. 제로 런타임 비용 - 컴파일 시간에 계산
2. 타입 정보 접근 - @sizeOf, @typeInfo 등
3. 조건부 컴파일 - 플랫폼별 코드
4. 제네릭 프로그래밍 - 타입을 매개변수로

---

17 장: 제네릭

제네릭은 타입을 매개변수로 받아 여러 타입에 작동하는 코드를 작성합니다.

```zig
const std = @import("std");

// 제네릭 함수
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

fn swap(comptime T: type, a: *T, b: *T) void {
    const temp = a.*;
    a.* = b.*;
    b.* = temp;
}

// 제네릭 스택
fn Stack(comptime T: type) type {
    return struct {
        const Self = @This();
        items: [100]T = undefined,
        count: usize = 0,

        pub fn push(self: *Self, item: T) !void {
            if (self.count >= 100) return error.StackOverflow;
            self.items[self.count] = item;
            self.count += 1;
        }

        pub fn pop(self: *Self) ?T {
            if (self.count == 0) return null;
            self.count -= 1;
            return self.items[self.count];
        }
    };
}

pub fn main() void {
    // 제네릭 함수 사용
    std.debug.print("max(i32, 5, 10) = {d}\n", .{max(i32, 5, 10)});
    std.debug.print("max(f64, 3.14, 2.71) = {d}\n", .{max(f64, 3.14, 2.71)});

    // swap 사용
    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\n교환 전: x={d}, y={d}\n", .{ x, y });
    swap(i32, &x, &y);
    std.debug.print(" 교환 후: x={d}, y={d}\n", .{ x, y });

    // 제네릭 스택 사용
    var int_stack = Stack(i32){};

    int_stack.push(10) catch {};
```

```
    int_stack.push(20) catch {};
    int_stack.push(30) catch {};

    std.debug.print("\n스택에서 pop:\n", .{});
    while (int_stack.pop()) |val| {
        std.debug.print("  {d}\n", .{val});
    }
}
```

출력:

```
max(i32, 5, 10) = 10
max(f64, 3.14, 2.71) = 3.14

교환 전: x=100, y=200
교환 후: x=200, y=100

스택에서 pop:
  30
  20
  10
```

핵심 포인트

1. 타입 매개변수 - comptime T: type 으로 타입 전달
2. 코드 재사용 - 하나의 구현으로 여러 타입 지원
3. 컴파일 시간 특수화 - 각 타입에 대해 최적화된 코드 생성
4. 타입 안전 - 컴파일 시간에 타입 검사

---

18 장: 메모리 관리

Zig 는 수동 메모리 관리를 제공하며, 할당자를 통해 메모리를 관리합니다.

```
const std = @import("std");

pub fn main() !void {
    // 스택 메모리
    var stack_arr: [10]i32 = undefined;
    for (&stack_arr, 0..) |*item, i| {
        item.* = @intCast(i * 2);
    }
    std.debug.print(" 스택 배열: {any}\n", .{stack_arr});

    // 힙 할당 (GeneralPurposeAllocator)
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // 단일 값 할당
    const ptr = try allocator.create(i32);
    defer allocator.destroy(ptr);
    ptr.* = 42;
```

```zig
    std.debug.print("\n힙 할당 값: {d}\n", .{ptr.*});

    // 슬라이스 할당
    const slice = try allocator.alloc(i32, 5);
    defer allocator.free(slice);

    for (slice, 0..) |*item, i| {
        item.* = @intCast((i + 1) * 10);
    }
    std.debug.print(" 힙 슬라이스: {any}\n", .{slice});

    // ArrayList 사용
    var list = std.ArrayList(i32).init(allocator);
    defer list.deinit();

    try list.append(1);
    try list.append(2);
    try list.append(3);
    std.debug.print("\nArrayList: {any}\n", .{list.items});
}
```

출력:

```
스택 배열: { 0, 2, 4, 6, 8, 10, 12, 14, 16, 18 }

힙 할당 값: 42
힙 슬라이스: { 10, 20, 30, 40, 50 }

ArrayList: { 1, 2, 3 }
```

핵심 포인트

1. 명시적 할당 - 메모리 할당이 명확히 보임
2. **defer** 로 해제 - 자원 누수 방지
3. 여러 할당자 - 용도에 맞는 할당자 선택
4. 가비지 컬렉션 없음 - 직접 메모리 관리

---

19 장: 테스팅

Zig 는 내장 테스트 프레임워크를 제공합니다.

```zig
const std = @import("std");
const expect = std.testing.expect;
const expectEqual = std.testing.expectEqual;

// 테스트할 함수
fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn divide(a: i32, b: i32) !i32 {
    if (b == 0) return error.DivisionByZero;
```

```zig
    return @divTrunc(a, b);
}

// 기본 테스트
test " 덧셈 테스트" {
    const result = add(2, 3);
    try expectEqual(@as(i32, 5), result);
}

test " 음수 덧셈" {
    try expectEqual(@as(i32, -1), add(2, -3));
    try expectEqual(@as(i32, 0), add(5, -5));
}

// expect 사용
test "expect 예제" {
    try expect(add(1, 1) == 2);
    try expect(10 > 5);
}

// 오류 테스트
test "0 으로 나누기" {
    try std.testing.expectError(error.DivisionByZero, divide(10, 0));
}

test " 정상 나눗셈" {
    const result = try divide(10, 2);
    try expectEqual(@as(i32, 5), result);
}

// 테스트 실행: zig test 19_testing.zig
pub fn main() void {
    std.debug.print(" 테스트 실행: zig test 파일이름.zig\n", .{});
}
```

테스트 실행:

```
zig test 19_testing.zig
```

출력:

```
All 5 tests passed.
```

핵심 포인트

1. **test** 블록 - 테스트 케이스 정의
2. **expect** 함수 - 조건 검증
3. 오류 테스트 - expectError 로 오류 발생 검증
4. 테스트 할당자 - 메모리 누수 감지

---

20 장: C 상호운용성

Zig 는 C 코드와 쉽게 통합됩니다.

```zig
const std = @import("std");
const c = @cImport({
    @cInclude("stdio.h");
    @cInclude("stdlib.h");
    @cInclude("string.h");
    @cInclude("math.h");
});

pub fn main() void {
    // C 타입 사용
    const a: c_int = 42;
    const b: c_long = 1000000;
    std.debug.print("c_int: {d}\n", .{a});
    std.debug.print("c_long: {d}\n", .{b});

    // C 수학 함수 호출
    const x: c.double = 2.0;
    const sqrt_x = c.sqrt(x);
    const pow_x = c.pow(x, 3.0);
    std.debug.print("\nsqrt({d}) = {d}\n", .{ x, sqrt_x });
    std.debug.print("pow({d}, 3) = {d}\n", .{ x, pow_x });

    // C 문자열
    const zig_str: [:0]const u8 = "Hello from Zig!";
    const c_str: [*c]const u8 = zig_str.ptr;
    _ = c.printf("C printf: %s\n", c_str);

    // strlen 사용
    const len = c.strlen(c_str);
    std.debug.print("strlen: {d}\n", .{len});
}

// C 로 내보내기
export fn zig_add(a: c_int, b: c_int) c_int {
    return a + b;
}

export fn zig_multiply(a: c_int, b: c_int) c_int {
    return a * b;
}
```

출력:

```
c_int: 42
c_long: 1000000

sqrt(2) = 1.41421...
pow(2, 3) = 8

C printf: Hello from Zig!
strlen: 15
```

핵심 포인트

1. **@cImport** - C 헤더 파일 직접 사용
2. **C** 타입 - c_int, c_long 등 호환 타입
3. **export** - C 에서 호출할 수 있는 함수 내보내기
4. 바인딩 불필요 - C 라이브러리 직접 사용

부록 A: Zig 치트 시트

변수

```
const x: i32 = 42;    // 상수
var y: i32 = 0;       // 변수
```

타입

```
i8, i16, i32, i64     // 부호 있는 정수
u8, u16, u32, u64     // 부호 없는 정수
f32, f64              // 부동소수점
bool                  // 불리언
[]const u8            // 문자열 (바이트 슬라이스)
```

제어 흐름

```
if (조건) { } else { }
switch (값) { ... }
while (조건) { }
for (배열) |항목| { }
```

함수

```
fn name(param: T) ReturnType { }
fn name(param: T) !ReturnType { }  // 오류 반환 가능
```

구조체

```
const Struct = struct {
    field: Type,
    pub fn method(self: *Struct) { }
};
```

옵셔널과 오류

```
?T                    // 옵셔널 타입
!T                    // 오류 유니온
value orelse default  // 옵셔널 기본값
```

```zig
try expression        // 오류 전파
catch |err| { }       // 오류 처리
```

메모리

```zig
const ptr = try allocator.create(T);
defer allocator.destroy(ptr);
const slice = try allocator.alloc(T, n);
defer allocator.free(slice);
```

부록 B: 일반적인 패턴

반복자 패턴

```zig
for (items) |item| { }
for (items, 0..) |item, idx| { }
```

오류 처리 패턴

```zig
const result = operation() catch |err| switch (err) {
    error.A => handleA(),
    error.B => handleB(),
    else => return err,
};
```

옵셔널 처리 패턴

```zig
if (optional) |value| {
    // 값 사용
} else {
    // null 처리
}
```

defer 패턴

```zig
const resource = acquire();
defer release(resource);
// resource 사용
```

Easy Zig - JW Lee 저

Zig 공식 문서: https://ziglang.org/documentation/master/